

DAA Lab

Name: Devansh Joshi

SAP ID: 590028189

Topic: Celebrity Problem

Celebrity Problem

Code:

```
C:\Users\devan\OneDrive\Desktop > g++ DEMO.cpp -std=c++11
#include <iostream>
using namespace std;

// Helper function to check if a person is using a dynamic 2D array
int isDynamic(int n, int k, int b) {
    return (k < b);
}

// Stack-based celebrity finder (Optimal)
int findCelebrity(int n, int k) {
    int* stack = (int*)calloc(n, sizeof(int));
    int top = -1;

    // Step 1: Push all people onto the stack
    for (int i = 0; i < n; i++) {
        stack[++top] = i;
    }

    // Step 2: Pop the candidates and eliminate one based on relationship
    while (top > 0) {
        int a = stack[top--];
        int b = stack[top];

        if (knows(a, b)) {
            // If 'a' knows 'b', 'a' cannot be celebrity. Push 'b' back.
            stack[++top] = a;
        } else {
            // If 'a' does not know 'b', 'b' cannot be celebrity. Push 'a' back.
            stack[++top] = b;
        }
    }

    // Step 3: Potential candidate remaining
    int candidate = stack[top];
    free(stack);

    // Step 4: Verify candidate
    for (int i = 0; i < n; i++) {
        if (i != candidate) {
            // Candidate must know everyone else except themselves
            if (knows(candidate, i) || !knows(i, candidate)) {
                return -1; // No celebrity exists
            }
        }
    }

    return candidate;
}

// Main function
int main() {
    int n = 4; // Dynamically set size

    // Initialize matrix for dynamic 2D array
    int** k = (int**)calloc(n, sizeof(int*));
    for (int i = 0; i < n; i++) {
        k[i] = (int*)calloc(n, sizeof(int));
    }

    // Initialize relationship matrix
    int matrix[n][n] = {
        {0, 0, 1, 0}, // Person 0 is celebrity
        {0, 0, 0, 0}, // Person 1 is celebrity
        {0, 0, 0, 0}, // Person 2 is celebrity
        {0, 0, 0, 0}
    };

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            k[i][j] = matrix[i][j];
        }
    }

    int result = findCelebrity(n, k);

    if (result == -1) {
        printf("No Celebrity Found.\n");
    } else {
        printf("Celebrity is Person ID: %d\n", result);
    }

    // Free dynamic memory
    for (int i = 0; i < n; i++) free(k[i]);
    free(k);

    return 0;
}
```

Output:

```
PS C:\Users\devan> cd "c:\Users\devan\OneDrive\Desktop\" ; if ($?) { gcc DEMO.c -o DEMO } ; if ($?) { .\DEMO }
Celebrity is Person ID: 2
PS C:\Users\devan\OneDrive\Desktop>
```

Time Complexity Graph:

